

Analysis: Cake

Eat Cake: Analysis

Small dataset

Let $f(N)$ be the minimum number of cakes that have to be eaten such that the total combined area of the eaten cakes is N . To compute $f(N)$, we can start by checking all possible sizes that we could use for the first cake that we eat. If the first eaten cake has an area of $A \times A$, then we need to eat remaining cake(s) with a total combined area of $N - A \times A$, which requires at least $f(N - A \times A)$ cakes to be eaten. Therefore, $f(N)$ can be recursively computed as follows:

```
f(N)
    if (N = 0)
        return 0
    ans = infinity
    for i in [1, sqrt(N)]
        ans = min(ans, f(N - i * i) + 1)
    return ans
```

The algorithm above is fast enough to solve the Small dataset.

Large dataset

We need to use [dynamic programming](#) (DP) / memoization to solve the large dataset. If the value of $f(N)$ has been recursively computed before, then the next time we need to know that value, we can simply use the previously computed value. Since the DP table of values has size $O(N)$, and computing each value of $f(N)$ takes $O(N^{0.5})$ time, the total time for this algorithm is $O(N^{1.5})$.

```
f(N)
    if (N = 0)
        return 0
    if (dp[N])
        return dp[N]
    dp[N] = infinity
    for i in [1, sqrt(N)]
        dp[N] = min(dp[N], f(N - i * i) + 1)
    return dp[N]
```

Alternative solution

There is another solution that does not even require a recursive strategy. From [Lagrange's four-square theorem](#), we know that $f(N)$ cannot be larger than 4. Therefore, for each k ($1 \leq k \leq 3$), we can simply have k nested loops of possible cake areas to determine whether k cakes are enough. If 3 cakes are still not enough, then the answer must be 4 (and we do not actually need to determine which cakes are used).

```
f(N)
    for i in [1, sqrt(N)]
        if (i * i = N)
```

```
    return 1
for i in [1, sqrt(N)]
    for j in [i, sqrt(N)]
        if (i * i + j * j = N)
            return 2
for i in [1, sqrt(N)]
    for j in [i, sqrt(N)]
        for k in [j, sqrt(N)]
            if (i * i + j * j + k * k = N)
                return 3
return 4
```

Since each layer of the triply nested loop adds $O(\mathbf{N}^{0.5})$ time, the total time for this algorithm is also $O(\mathbf{N}^{1.5})$.

Analysis: Catch Them All

Catch Them All: Analysis

Small dataset

We can start by computing the shortest distance between each pair of locations using the [Floyd-Warshall](#) algorithm. We will use $\text{dis}[i, j]$ to represent the shortest distance between locations i and j .

Let $\text{dp}[K, L]$ be the expected time needed to catch K Codejamons when starting from location L . Then we can use a dynamic programming algorithm with the following state transition equation:

```
if (K == 0):
    dp[K, L] = 0;
else:
    dp[K, L] =  $\sum_{i=1}^N (\text{dp}[K-1, i] + \text{dis}[L, i]) / (N-1)$ .
```

The algorithm above takes $O(N^2P)$ time, which is fast enough to solve the Small dataset.

Large dataset

We can find that for each $\text{dp}[K, L]$, the answer is a linear expression of $\text{dp}[K-1, i]$ when $K \neq 0$. So, we can rewrite the state transition equation as the product of a matrix and a column vector, as shown below.

Let $S[i] = \sum_{j \neq i} (\text{dis}[i, j])$.

$$\begin{array}{c|c|c|c|c}
 +-----+ & +-----+ & & +-----+ & \\
 | \text{dp}[K, 1] | & | 0, 1/(N-1), \dots, 1/(N-1), S[1]/(N-1) | & & | \text{dp}[K-1, 1] | & \\
 | \text{dp}[K, 2] | & | 1/(N-1), 0, \dots, 1/(N-1), S[2]/(N-1) | & & | \text{dp}[K-1, 2] | & \\
 | \dots | & | \dots, \dots, \dots, \dots, \dots | & * & | \dots | & \\
 | \text{dp}[K, N] | & | 1/(N-1), 1/(N-1), \dots, 0, S[N]/(N-1) | & & | \text{dp}[K-1, N] | & \\
 | 1 | & | 0, 0, \dots, 0, 1 | & & | 1 | & \\
 +-----+ & +-----+ & & +-----+ &
 \end{array}$$

Let F_K denote the column vector of $\text{dp}[K, i]$, and let A denote the transition matrix. Then we have $F_K = A * F_{K-1} = A^K * F_0$.

With the approach above, we can use [exponentiation by squaring](#) to accelerate the computation of A^K . This gives us an $O(N^3 \log P)$ algorithm which can solve the Large dataset.

Other solutions

Let Pr_t be the probability of being at location 1 after catching t Codejamons. Initially, $\text{Pr}_0 = 1$. Since at any time, the probabilities of being at locations 2, 3, ..., N (let's call them the "other locations") are the same, we can calculate the probability of the next Codejamon appearing at location 1 by multiplying the probability of being at any of the other locations by the probability of location 1 being chosen. Therefore, we have $\text{Pr}_t = (1 - \text{Pr}_{t-1}) / (N-1)$.

After computing the values of Pr_i for $i = 1, 2, \dots, P-1$, the answer to the problem is:

$\sum (\text{Pr}_i * (\text{expected distance from location 1}) + \sum (1 - \text{Pr}_i) / (N-1) * (\text{expected distance from location } j) \text{ for } j = 2, 3, \dots, N) \text{ for } i = 0, 1, \dots, P-1$.

Note: the expected distance from location i equals $\sum \text{dis}[i, j] / (N-1)$ for $j = 1, 2, \dots, N$.

We might not have enough time to compute all P values of the sequence Pr_i , but one may notice that this sequence converges quickly (except for $N=2$, which we can handle separately). Intuitively, as the game

progresses, the probabilities of you being at each of the N locations become more equal. For example, if $N=4$, the first few values for Pr_t are 1, 0, 0.333, 0.222, 0.259, 0.247, 0.251, 0.250, ... Once this value gets very close to $1/N$, after some threshold like $i = 100$ (depending on the numerical error allowed), we can simply approximate $Pr_i = 1/N$ for all i larger than the threshold. Then $i = 100, 101, \dots, P-1$ in the previous summation can be replaced by a multiplication: $(P-100) * \sum(\text{expected distance from location } j) / N$ for $j = 1, 2, \dots, N$.

The time complexity for the above algorithm is $O(N^3 + C)$, where C depends on the numerical error allowed.

Using the sequence of Pr_t values, it is also possible to calculate the exact answer. Let's make another sequence $A_i = Pr_i - 1/N$. This sequence converges to 0 and is a geometric progression because:

- $Pr_i = (1 - Pr_{i-1}) / (N-1)$
- $A_i + 1/N = (1 - A_{i-1} - 1/N) / (N-1)$
- $A_i + 1/N = -A_{i-1} / (N-1) + 1/N$
- $A_i = -A_{i-1} / (N-1)$.

$\sum A_i$ for $i = 0, 1, \dots, P-1$ can be calculated using the formula for the sum of a [geometric series](#). With $\sum A_i$, we can obtain $\sum Pr_i = 1/N * P + \sum A_i$, and ultimately the answer. The time complexity for the above algorithm is $O(N^3)$.

Analysis: Dance Battle

Dance Battle: Analysis

Even before solving the Small dataset, we need to reduce the number of options available to us, because the Delay action could allow a dance battle to go on forever! A critical initial insight is that we can use Delay to sort the opponents and face them in whatever order we want: we can delay against everyone else until we face our first desired opponent, then take some other non-delay action on that opponent, then keep delaying until we face our second desired opponent, and so on.

Small dataset

Once we know that we can use Delay to sort, the Small dataset's problem space allows us to use brute force. We will consider the worst case, $N = 5$. We can choose one of $5!$ possible orders in which to face the opponents; for each opponent, we choose one of the three other actions (Dance, Truce, or Recruit). That is a total of $5! \times 3^5 =$ only 29160 possible scenarios. We can simulate each of them to make sure that our energy does not drop below 1, and our honor does not drop below 0. Then, we take the maximum honor value among all valid scenarios. Each simulation takes linear time, so the overall time complexity of this strategy is $O(N! \times 3^N \times N)$.

Large dataset

The brute force method above will not scale to 1000 opponents. Let us devise an alternate strategy. For starters, we can observe that we cannot use the Recruit action until we have defeated at least one opponent (and gained a point of honor) by using Dance. So, if our starting energy level does not let us defeat the opponent with the lowest dancing skill, then our best option is to use Truce on everyone and finish with 0 honor.

Suppose that we can defeat at least one opponent by using Dance. Then we have no reason not to choose the opponent with the lowest dancing skill, since all opponents give the same amount of honor when defeated via Dance. Moreover, we may as well continue dancing against the weakest remaining opponent as long as we have the energy to do so.

What happens when we do not have enough energy to Dance against any remaining opponent? We can either use Truce to send everyone else away, or Recruit some opponent. If we are going to recruit someone, we should pick the opponent with the highest energy, since the cost of recruiting any opponent is the same. But how do we decide whether to use Recruit or Truce?

As long as there are at least two opponents remaining, it cannot hurt us to recruit the one with the most energy, because after that, we can surely defeat at least the one with the least energy. Even if we can defeat only one, and that one has the same skill as the opponent we recruited, we have lost one honor and gained one honor, and we have had no net change in energy, so we are no worse off than if we had used Truce to remove the same two opponents.

So, we can sort the opponent list from lowest to highest energy, and start with one pointer at each end. Setting aside the case in which we cannot defeat anyone: first, we move our left pointer forward, defeating opponents by dancing until we no longer can. Then, as long as there are at least two step, recruiting the strongest opponent and gaining energy. We repeat this until the pointers meet, or we have one opponent left that we cannot defeat (in which case we should

use Truce instead of wasting a point of honor by using Recruit). Whatever amount of honor we have at that point is our answer.

This algorithm has an $O(N \log N)$ sorting step followed by an $O(N)$ execution step. Other $O(N)$ execution steps are possible; for example, we can notice that the algorithm above will terminate with the pointers either zero or one opponent apart, so we can use partial sums and binary search to directly find that point.

Analysis: Kicksort

Kicksort: Analysis

In Quicksort, the best pivots at any stage are the ones that are closest to the *median* of the current list (although we would have no way of knowing which ones those are without doing some additional work before choosing!) This problem hinges on a misinterpretation of that idea, in which we take pivots from near the middle *index* of the list instead. Unless our sequence happens to be already sorted or close to sorted, this is no better than choosing an arbitrary index!

A Small-only approach

The sequences in the Small dataset can have between 2 and 4 elements. There are only $2! + 3! + 4! = 32$ different permutations that meet these criteria. This enables some strategies that do not usually work in Kickstart. For example, we can pre-solve each case by hand, or use nested conditional statements that take advantage of the small size of the sequence. But we recommend using one of the two approaches below instead.

A quadratic simulation approach

The most straightforward general approach is to actually run the Kicksort algorithm on the given sequence, and see whether it ever picks a pivot that is *not* worst-case. One possible complication is figuring out how to have this recursive algorithm pass information up the call chain if it does find a non-worst-case pivot; one option is to have it throw an exception that can then be caught. We must also be careful not to overflow the stack with recursive calls, since there can be N of them; one way to avoid this is to use our language's provided way to change the system's stack size. (Warning: This sort of system tweaking can be dangerous in real-world applications, which tend to avoid deep recursive strategies like this one.)

Quicksort is famously $O(N^2)$ when it consistently picks worst-case pivots, and that is what many of our simulations will do, so this strategy takes $O(N^2)$ time. This is still fast enough to solve the Large dataset, but can we do better?

A linear approach

First of all, let us consider different sequences of the same length that have "YES" answers — that is, the ones for which Kicksort will always pick worst-case pivots. Even though the pivots themselves may have different values, Kicksort will always use the same *indexes*, in the same order, as pivots. For a sequence of length 6, for example, it will pick index 2 as a pivot, and then divide the other values into one empty list (which is unimportant since no pivot is chosen from it) and one list of length 5 containing the remaining values. (It turns out not to matter for our purposes whether that list is the "low" or "high" one.) Then it will pick index 2 from that list of length 5, which corresponds to index 3 in the original list. If we continue to trace such a case, we find that the indexes chosen from the original list will be 2, 3, 1, 4, 0, 5. That is, we are starting at index $\text{floor}(N - 1) / 2$, then jumping 1 to the right, then 2 to the left, then 3 to the right, and so on. This does not depend at all on the values in the list!

A similar pattern holds for lists of odd length, although in that case, the first jump is 1 to the left. Knowing this, we can visit the indexes of our pivots in order, without doing any simulation. It is

not too hard to implement the pattern of changing direction and adding a distance of 1 with each new jump.

Each time we visit a new index, we check whether it holds either the lowest or highest value that has not already been used. If this continues all the way through the last index, we have a "YES" case. However, if we ever encounter a value that does not satisfy those conditions, then we have a "NO" case and we can stop.

At this point, we can take advantage of the fact that our sequences are permutations of numbers from 1 to **N**. (Of course, in real life, we could "sort" such sequences in constant time, without ever reading them, if we knew the length in advance!) We know at the outset that our lowest unused value is 1 and our highest unused value is **N**. When our current value matches this lower bound, we increment it by 1, because we already know that the next lowest value is exactly 1 more than the previous lowest; similarly, when our pivot matches the upper bound, we increment our upper bound by 1. We only need to keep track of these two values as we go.

Since we potentially have to bounce around the entire sequence, this strategy takes $O(N)$ time. It is unusual for a problem about sorting to have a solution that takes less than $O(N \log N)$ time, but in this case, it is a consequence of having restricted our sequences to permutations.